A sound pure type system with ranks as sorts and Type-in-Type

Pavel Sokolov
HSE University, Laboratory of Theoretical Computer Science

Alexander Kuklev
Radboud Univ. Nijmegen, Department of Software Science

October 12, 2026

Abstract

We present a dependently typed pure type system which contains a circular axiom $s : s$ for a certain sort s and is nonetheless strongly normalizing. Girard's paradox is avoided because our sorts are not closed under a Π-former: function types are placed in sorts according to their *rank*. We prove strong normalisation of a system and provide a bare-bones set-theoretical model for it. A normalisation proof is surprisingly simple (modulo general metatheoretical properties of pure type systems) and has been fully formalized in vanilla Rocq (formerly Coq) without any additional axioms.

1 Introduction

Pure type systems (PTSs) are a family of minimalistic typed λ-calculi including grammar formers for variables $x \in \mathsf{Var}$, abstraction $\lambda(x : T).t$, application $t\,t'$, types of dependent functions $\Pi(x : T).U$ (where U might depend on x) and *sorts* $s \in \mathcal{S}$ which act like "types of types". In contrast with many other type systems, there is no syntactical distinction between terms and types — they are defined by a single grammar; an expression can act both as a term and as a type depending on a side where it stands next to the colon in the typing judgement $\Gamma \vdash _ : _$. A single PTS is determined by two relations:

- *axiom* relation $\mathsf{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ determines "to which sort(s) a given sort belongs as an element"; instead of $(s, s') \in \mathsf{Ax}$ it is customary to write simply $s : s'$.
- *rule* relation $\mathsf{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ places types of functions $\Pi(x : T).U$ into sorts accounting for the sorts where T and U are situated.

$$\textsc{Ax}\ \frac{}{\Gamma \vdash s : s'}(s, s') \in \mathsf{Ax} \qquad \textsc{Rule}\ \frac{\Gamma \vdash T : s_T \qquad \Gamma, x : T \vdash U : s_U}{\Gamma \vdash \Pi(x : T).\, U : s}(s_T, s_U, s) \in \mathsf{Rule}$$

$$\textsc{Var}\ \frac{}{\Gamma \vdash x : T}x : T \in \Gamma \qquad \textsc{Abs}\ \frac{\Gamma \vdash T : s_T \qquad \Gamma, x : T \vdash u : U}{\Gamma \vdash \lambda(x : T).u : \Pi(x : T).U}$$

$$\textsc{App}\ \frac{\Gamma \vdash u : \Pi(x : T).U \qquad \Gamma \vdash t : T}{\Gamma \vdash u\, t : U[t/x]} \qquad \textsc{Conv}\ \frac{\Gamma \vdash t : T \qquad \Gamma \vdash U : s \qquad T =_\beta U}{\Gamma \vdash t : U}$$

Figure 1: Typing rules of pure type systems

Systems from Barendregt's lambda cube fit naturally into the framework: for example, simply typed lambda calculus (STLC) can be viewed as a pure type system with a single sort $*$, no axioms and a single rule $(*, *, *)$. In general, however, a pure type system is dependently typed since Π allows the codomain type to depend on the value of an argument — Coquand's Calculus of Constructions (CoC) sits comfortably at the top of lambda cube and is modeled as a pure type system with two sorts $\{*, \Box\}$, a single axiom $* : \Box$ and four rules corresponding to simple functions $(*, *, *)$, polymorphic functions $(\Box, *, *)$, type operators $(\Box, \Box, \Box)$ and dependent types $(*, \Box, \Box)$.

As is known, both STLC and CoC are strongly normalizing and sound with respect to their set-theoretical semantics. A classical result shows that it is not true for arbitrary PTS.

Theorem (Girard'72)**.** *The following PTS called System U is not normalizing:*

$$\mathcal{S} = \{*, \Box, \triangle\}, \quad \mathsf{Ax} = \{(*, \Box), (\Box, \triangle)\},$$
$$\mathsf{Rule} = \{(*, *, *), (\Box, *, *), (\triangle, *, *), (\Box, \Box, \Box), (\triangle, \Box, \Box)\}$$

Futhermore, the offending term has type $\Pi(P : *).P$*.*

This result is dubbed "Girard's paradox" since it renders System U inconsistent with respect to its intended logical semantics: inhabitedness of $\Pi(P : *).P$ would mean that every proposition is provable. Thus any system where System U embeds is inconsistent, too; in particular, an original version of Martin-Löf's Intuitionistic Type Theory embeds the following PTS known as "Type-in-Type" which is inconsistent due to Girard's paradox:

$$\mathcal{S} = \{*\}, \quad \mathsf{Ax} = \{* : *\}, \quad \mathsf{Rule} = \{(*, *, *)\}.$$

For a pure type system with usual set-theoretical semantics, it is natural to expect that a sort of *small types* $*$ is closed under Π, thus we cannot forbid the rule $(*, *, *)$; then, a circular axiom $* : *$ has to go. Long story short, it

motivates an introduction of a *universe hierarchy* $\mathcal{S} = \{\mathcal{U}_k\}_{k\in\mathbb{N}}$ with axioms $\mathcal{U}_k : \mathcal{U}_{k+1}$ and rules $(\mathcal{U}_k, \mathcal{U}_l, \mathcal{U}_{k\vee l})$ for each k, l; when interpreted in sets, $[\![\mathcal{U}_k]\!]$ is taken to be the k-th inaccessible cardinal.

Observing this, we took interest in the following two questions:

- If we were to drop the rule $(*, *, *)$ and keep the axiom $* : *$ instead, can we find a nontrivial consistent pure type system?
- Is there a consistent pure type system with total relations Ax and Rule whose natural set-theoretical semantics do not involve assumption of inaccessible cardinals?

Turns out that, for both questions, answer is "yes". Furthermore, both yes-s are witnessed by a single PTS, which we dub PTS-rk:

$$\mathcal{S} = \{\mathsf{rk}_r\}_{r\in\mathbb{N}} \quad \mathsf{Ax} = \{(\mathsf{rk}_r, \mathsf{rk}_0)\}_{r\in\mathbb{N}} \quad \mathsf{Rule} = \{(\mathsf{rk}_r, \mathsf{rk}_s, \mathsf{rk}_{(r+1)\vee s})\}_{r,s\in\mathbb{N}}$$

(Naming "rk" comes from *rank*, since rules compute precisely the rank of functions, which is the maximal nesting of Π in the domain type.)

Indeed, there is a circular axiom $\mathsf{rk}_0 : \mathsf{rk}_0$, and System U is not embeddable since rk_r is not closed under Π for any r. Also, as we will see in Section 3, our PTS-rk is strongly normalizing and has a set-theoretical model in ZFC despite an intuition that $\mathsf{rk}_r : \mathsf{rk}_0$ might "feel wrong". Modulo general metatheory of pure type systems laid out in Section 2, a normalisation proof turned out to be suspiciously simple, even more so in comparison with normalisation proofs of other dependently typed systems. To dispel our own doubts, we formalized a proof of normalisation in Rocq (formerly Coq) without using any additional axioms, yielding a fully constructive proof; our journey in formalization is summarised in Section 3.1. We provide a closing summary and some pointers on possible extensions of the system in Section 7.

2 Metatheory of pure type systems

For ease of presentation of our results, in Sections 2-3 we take a Barendregt convention which assumes that all bound variables are distinct from free variables and from each other; furthermore, α-equivalence (renaming of bound variables) is assumed as a trivial equality on terms. Nevertheless, our formalization takes variable treatment seriously; see Section 3.1 for more details.

2.1 PTS as a rewriting system

In Fig. 1, a rule of *conversion* (Conv) mentions *definitional equality* of types $T =_\beta U$. It is defined as a reflexive, symmetric, transitive closure of a one-step reduction.

Definition 1 (One-step reduction)**.** Relation of a one-step reduction $t \to_\beta t'$ is the smallest relation satisfying the following rules:

$$\frac{T \to_\beta T'}{\Pi(x:T).U \to_\beta \Pi(x:T').U} \qquad \frac{U \to_\beta U'}{\Pi(x:T).U \to_\beta \Pi(x:T).U'}$$

$$\frac{T \to_\beta T'}{\lambda(x:T).t \to_\beta \lambda(x:T').t} \qquad \frac{t \to_\beta t'}{\lambda(x:T).t \to_\beta \lambda(x:T).t'} \qquad \frac{t \to_\beta t'}{t\,u \to_\beta t'\,u}$$

$$\frac{u \to_\beta u'}{t\,u \to_\beta t\,u'} \qquad \textsc{Beta}\ (\lambda(x:T).t)\,u \to_\beta t[u/x]$$

In essence, actual reduction happens only in rule Beta; other rules simply state the we can freely reduce any subterm.

The following result will be greatly useful in our development. It was proven many times over for different similar systems; we briefly revisit the proof since previous presentations usually did not allow reductions in the type of argument T in $\lambda(x:T).t$.

Theorem 1 (Characterization of $(=_\beta)$)**.** *In arbitrary PTS,*

$$t =_\beta u \quad \Leftrightarrow \quad \exists v.\ t \to_\beta^* v \wedge u \to_\beta^* v,$$

where $(\to_\beta^*)$ *is reflexive transitive closure of* $(\to_\beta)$*.*

Proof. Note that it is enough to prove that $(\to_\beta^*)$ has a diamond property — for each t_0, t_1, t_2 such that $t_0 \to_\beta^* t_1$ and $t_0 \to_\beta^* t_2$ there exists a t' such that $t_1 \to_\beta^* t'$ and $t_2 \to_\beta^* t'$:

$$\begin{array}{ccc} t_0 & \longrightarrow & t_1 \\ \downarrow & & \vdots \\ t_2 & \dashrightarrow & t' \end{array}$$

Then, any "mountains" in the path $t =_\beta u$ can be switched into "valleys".

Now, diamond property is proven by standard technique of parallel reduction which is defined as the smallest relation satisfying:

$$t \rightrightarrows t \qquad \frac{T \rightrightarrows T' \qquad U \rightrightarrows U'}{\Pi(x:T).U \rightrightarrows \Pi(x:T').U'} \qquad \frac{T \rightrightarrows T' \qquad t \rightrightarrows t'}{\lambda(x:T).t \rightrightarrows \lambda(x:T').t'}$$

$$\frac{t \rightrightarrows t' \qquad u \rightrightarrows u'}{t\,u \rightrightarrows t'\,u'} \qquad \frac{t \rightrightarrows t' \qquad u \rightrightarrows u'}{(\lambda(x:T).t)\,u \rightrightarrows t'[u'/x]}$$

Intuitively, while $(\to_\beta)$ is a single β-reduction somewhere in the term and $(\to_\beta)^*$ is a sequence of reductions, $(\rightrightarrows)$ is several simultaneous reductions immediately available in the term. Notice, then, that $(\to_\beta) \subseteq (\rightrightarrows) \subseteq (\to_\beta^*)$; by monotonicity of reflexive transitive closure, $(\to_\beta^*) \subseteq (\rightrightarrows^*) \subseteq (\to_\beta^*)^* = (\to_\beta^*)$, so $(\rightrightarrows^*) = (\to_\beta^*)$.

Thus it is enough to prove diamond property of $(\rightrightarrows^*)$. Diamond property is preserved under reflexive transitive closure, so it is enough to prove diamond propery of $(\rightrightarrows)$, which holds by tedious case analysis. □

Setting aside practical applications of this theorem (to algorithmically check equivalence of terms, we can now search for common reduction result), it will prove useful already in our proof development. For example, the following "equality inversion" lemmas are direct corollaries:

Corollary 1 (Equality inversion)**.** *If $s =_\beta s'$, then $s = s' \in \mathcal{S}$; if $\Pi(x : T).U =_\beta \Pi(x : T').U'$, then $T =_\beta T'$ and $U =_\beta U'$.*

They will become necessary in proving subject reduction, or type preservation. But first, we will show some structural properties of a typing judgement given in Fig. 1.

2.2 Properties of a typing judgement

Again, they surely have been proven before for PTS, but we slightly differ in the formulation of typing rules in important places so it's better to recheck that these hold.

Lemma 1 (Weakening)**.** *The following rule is admissible:*

$$\frac{\Gamma, \Delta \vdash t : T}{\Gamma, x : U, \Delta \vdash t : T}.$$

Proof. By structural induction on the typing derivation. □

Lemma 2 (Substitution)**.** *The following rule is admissible:*

$$\frac{\Gamma, x : U, \Delta \vdash t : T \quad \Gamma \vdash u : U}{\Gamma, \Delta[u/x] \vdash t[u/x] : T[u/x]}.$$

Proof. By structural induction on the typing derivation for t. For variables in Δ, it is necessary that both Δ and T get substitutions of u for x so Var is still applicable; for places in t where x used to be we need to apply weakening lemma to the typing derivation for u exactly $|\Delta|$ times. In the case of Conv it is important that definitional equality of types is preserved by substitution (thanks to characterization of $(=_\beta)$). □

In addition, a family of so-called *inversion lemmas* stating that type of well-typed term is largely determined by shape of a term are also proven by straightforward induction on typing derivation (each one is proven separately, no mutual induction involved):

Lemma 3 (Inversion lemmas)**.**

$$\Gamma \vdash s : T \Rightarrow \exists s'.(s, s') \in \mathsf{Ax} \wedge T =_\beta s' \tag{1}$$

$$\Gamma \vdash \Pi(x : T).U : V \Rightarrow \exists s_T, s_U, s. \begin{cases} (s_T, s_U, s) \in \mathsf{Rule} \\ \Gamma \vdash T : s_T \\ \Gamma, x : T \vdash U : s_U \\ V =_\beta s \end{cases} \tag{2}$$

$$\Gamma, x : T, \Delta \vdash x : U \Rightarrow T =_\beta U \tag{3}$$

$$\Gamma \vdash \lambda(x : T).t : V \Rightarrow \exists s, U. \begin{cases} \Gamma \vdash T : s \\ \Gamma, x : T \vdash t : U \\ V =_\beta \Pi(x : T).U \end{cases} \tag{4}$$

$$\Gamma \vdash t\, u : V \Rightarrow \exists U, T. \begin{cases} \Gamma \vdash t : \Pi(x : U).T \\ \Gamma \vdash u : U \\ V =_\beta T[u/x] \end{cases} \tag{5}$$

To progress further, we need to make certain assumptions on the PTS we have in question.

Definition 2 (Functional PTS)**.** A pure type system is called *functional* if $\mathsf{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ and $\mathsf{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ are functional relations, that is,

$$(s : s') \wedge (s : s'') \Rightarrow s' = s'' \quad \text{and} \quad (s_1, s_2, s) \in \mathsf{Rule} \wedge (s_1, s_2, s') \in \mathsf{Rule} \Rightarrow s = s'$$

Definition 3 (Total PTS)**.** A pure type system is called *total* if $\mathsf{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ and $\mathsf{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ are total relations, that is,

$$\forall s. \exists s'.(s, s') \in \mathsf{Ax} \quad \text{and} \quad \forall s_1, s_2. \exists s.(s_1, s_2, s) \in \mathsf{Rule}$$

Theorem 2 (Uniqueness of typing)**.** *In a functional PTS, the following rule is admissible:*

$$\frac{\Gamma \vdash t : T \quad \Gamma \vdash t : T'}{T =_\beta T'}.$$

Proof. By structural induction on t and inversion of given typing derivations.

- $t = s \in \mathcal{S}$: by functionality of Ax.
- $t = \Pi(x : T).U$: inversion of typing gives a pair of (possibly different) sorts for T and another pair for U. By induction we get that ranks inside pairs are definitionally equal, which by equality inversion gives strictly equal sorts. Then what is needed holds by functionality of Rule.
- Case for $t = x \in \mathsf{Var}$ holds trivially.
- $t = (\lambda(x : T).u)$: by induction hypotheses and by congruence of $(=_\beta)$.
- $t = u\, v$: by typing inversion and by induction hypothesis for u we get $u : \Pi(x : T_1).U_1$ and $\Pi(x : T_1).U_1 =_\beta \Pi(x : T_2).U_2$. We need $U_1[v/x] =_\beta U_2[v/x]$ which we get from equality inversion and preservation of def.eq. under substitution.

□

In our proof of normalisation, we're going to rely on yet another property: if there is a derivation for $t : T$, then T is itself typeable with some sort. However, it doesn't hold in our current formulation of typing rules: in $\Gamma \vdash x : T$, nothing prevents a context Γ to contain an entry $(x : T')$ where T' is not typeable. Thus we need an additional assumption.

Definition 4 (Context well-formedness)**.** Context Γ is *well-formed* (written $\vdash \Gamma$) if every type in it is typed by some sort. Formally, set of well-formed contexts is the least set closed by the following rules:

$$\frac{}{\vdash \cdot} \quad \text{and} \quad \frac{\vdash \Gamma \quad \Gamma \vdash T : s}{\vdash \Gamma, x : T}$$

(where $\cdot$ denotes an empty context.)

Usually, when presenting a dependently typed system, context well-formedness is added as a premise to typing rules which would otherwise be premise-free (such as Ax and Var in a PTS). Here we instead provide it as an additional assumption only when necessary; in addition to leading to simpler metatheory (we avoid an otherwise mutually inductive definition of $(\vdash _)$ and $(_ \vdash _ : _)$), this is closer to how decision procedures for typing judgements are implemented.

Theorem 3 (Typing yields correct type)**.** *In a total PTS, if* $\Gamma \vdash t : T$ *and* $\vdash \Gamma$*, then there exists* $s \in \mathcal{S}$ *such that* $\Gamma \vdash T : s$*.*

Proof. By structural induction on typing derivation.

- Ax, Rule: by totality of Ax.
- Var: follows from $\vdash \Gamma$.
- Abs: by totality of Rule.
- App: $\Gamma \vdash t\,u : T[u/x]$. From induction hypothesis for t we know that $\Gamma \vdash t : \Pi(x : U).T : s$ for some s, then by typing inversion for $\Pi(x : U).T$ we get that T is typable with some sort. Then what is needed follows from substitution lemma.
- Conv: what is needed is present in the premise.

□

Totality is not a strictly necessary requirement — similar true propositions can be stated for non-total PTS, for example in CoC typing yields type which is either itself typable or equal to $\Box$. But PTS-rk is indeed total so the theorem above is enough for our purposes.

Theorem 4 (Subject reduction)**.** *In a total PTS, the following rule is admissible:*

$$\frac{\Gamma \vdash t : T \quad \vdash \Gamma \quad t \to_\beta t'}{\Gamma \vdash t' : T}$$

Proof. By structural induction over derivation of $t \to_\beta t'$. Inversion of a typing judgement yields all necessary info, albeit because of it types are now given only up to definitional equality so we have to apply conversion rule in each case. Crucially, it requires the type we convert into to be typable by a sort, so that's where we use Theorem 3.

The only tricky case are reductions of a domain type in $\Pi(x : T).U$ and $\lambda(x : T).t$. Let us consider the case of the following reduction:

$$\lambda(x : T).t \to_\beta \lambda(x : T').t.$$

In order to complete this case, we have to prove that $\Gamma, x : T' \vdash t : U$ given that $\Gamma, x : T \vdash t : U$. By appropriately strengthening the statement, we get that it is enough to prove admissibility of the following rule called *context reduction*:

$$\frac{\Gamma, x : U, \Delta \vdash t : T \quad \vdash \Gamma, x : U, \Delta \quad U \to_\beta U'}{\Gamma, x : U', \Delta \vdash t : T}$$

The proof is, again, by induction on the typing derivation. The only non-trivial case is with VAR rule used on x; it is taken care of by appropriate use of CONV rule to convert $x : U'$ to $x : U$. To use conversion, we have to prove that U is well-typed; thankfully, it follows from $\vdash \Gamma, x : U, \Delta$.

Note that context reduction is admissible in arbitrary PTS! We don't need it anywhere else, though, so we didn't state it as a separate lemma. □

Characterization of definitional equality, uniqueness of typing and subject reduction together imply the following corollary we're going to use a few times in our normalisation proof:

Corollary 2 (Type stability)**.** *In a total functional PTS, the following holds:*

$$\begin{cases} \Gamma \vdash t : T \\ \Gamma \vdash t' : T' \\ \vdash \Gamma \\ t =_\beta t' \end{cases} \quad \Rightarrow \quad T =_\beta T'.$$

2.3 Properties of strongly normalizing terms

We're almost done with classical results! What is left now is to state which terms are strongly normalizing and collect properties of s.n. terms which allow our soundness proof to go through.

Informally, a term is strongly normalizing iff there are no infinite reduction chains starting from it. In constructive setting and in formalization, the following definition is a bit better:

Definition 5 (Strongly normalizing term)**.** Set of s.n. terms is the smallest set sn in which the following rule is valid:

$$\frac{\forall t', (t \to_\beta t') \Rightarrow t' \in \mathsf{sn}}{t \in \mathsf{sn}}.$$

In other words, a term is strongly normalizing iff all possible reductions from it are strongly normalizing.

The following properties are then provable by definition of sn and by structural induction on available derivations of sn for terms in question:

Lemma 4 (Structural properties of sn)**.**

$$x, s \in \mathsf{sn} \qquad \frac{T \in \mathsf{sn} \qquad U \in \mathsf{sn}}{\Pi(x:T).U \in \mathsf{sn}} \qquad \frac{T \in \mathsf{sn} \qquad t \in \mathsf{sn}}{\lambda(x:T).t \in \mathsf{sn}} \qquad \frac{t\,u \in \mathsf{sn}}{t \in \mathsf{sn}} \qquad \frac{t[u/x] \in \mathsf{sn}}{t \in \mathsf{sn}}$$

Application, of course, is the only nontrivial case. In case of β-reduction, strong normalisation of both $(\lambda(x:T).t)$ and u doesn't imply anything about normalisation behaviour of $t[u/x]$: consider a classical example from untyped lambda calculus, $\omega = (\lambda(x:T).x\,x)$ and $\Omega = \omega\,\omega$; while obviously not typable, ω is strongly normalizing, while Ω decidedly is not.

Situation changes when a head of application is not an abstraction at all. Thus we arrive at the definition of an *atomic term*.

Definition 6 (Atomic term)**.** Set of atomic terms is the smallest set satisfying the following conditions:

1. s, $\Pi(x:S).T$ and y are atomic for all variables x,y, sort s and types S,T;
2. If t is atomic, then $t\,u$ is also atomic.

Atomic terms are useful precisely because their strong normalisation survives application:

Lemma 5. *If $t, u \in \mathsf{sn}$ and t is atomic, then $t\,u \in \mathsf{sn}$.*

Proof. By induction on derivations of sn for t and u. Consider possible redexes in $t\,u$: $t\,u$ is not itself a redex since t is atomic and hence not a lambda; the only possible redexes are either inside t or inside u. Applying corresponding induction hypotheses, we reach the desired result. The only caveat is that, for induction to go through, we need to check that atomicity of t is preserved by reduction; this is indeed true by induction on derivation of atomicity for t. □

A dual view on strong normalisation of application is the following: if we already know that $t[u/x] \in \mathsf{sn}$, then the most tricky case is already taken care of.

Lemma 6.

$$\begin{cases} t[u/x]u_1 \dots u_n \in \mathsf{sn} \\ U, u \in \mathsf{sn} \end{cases} \Rightarrow (\lambda(x:U).t)uu_1 \dots u_n \in \mathsf{sn}.$$

Proof. By induction on n.

- $n = 0$. We need to prove that if $t[u/x], u, U \in \mathsf{sn}$, then $(\lambda(x : U).t)u \in \mathsf{sn}$. Note that $t[u/x] \in \mathsf{sn}$ implies $t \in \mathsf{sn}$; then, by structural induction on derivations of sn for t, u, U we get what we want by considering all possible reductions.

- $n \to (n+1)$. By induction hypothesis, it is enough to show the following:

$$\begin{cases} (\lambda(x : U).t)uu_1 \dots u_n \in \mathsf{sn} \\ t[u/x]u_1 \dots u_{n+1} \in \mathsf{sn} \end{cases} \Rightarrow (\lambda(x : U).t)uu_1 \dots u_n u_{n+1} \in \mathsf{sn}.$$

  This is true by structural induction on available evidence.

□

3 Strong normalisation of PTS-rk

Theorem 5 (Strong normalisation of PTS-rk)**.**

$$\Gamma \vdash t : T \wedge \vdash \Gamma \Rightarrow t \in \mathsf{sn}.$$

Proof. By induction on the typing derivation. Case Conv is trivial; due to Lemma 4, cases Ax, Rule, Var and Abs present no problem as well. The only problematic case is application:

$$\begin{cases} \Gamma \vdash u : \Pi(x : T).U \\ \Gamma \vdash t : T \\ \vdash \Gamma \\ t, u \in \mathsf{sn} \end{cases} \Rightarrow u\,t \in \mathsf{sn}$$

Now, here is the part where stratification according to ranks comes in useful. As typing yields correct type, there is some r such that $\Gamma \vdash \Pi(x : T).U : \mathsf{rk}_r$. Let us explicitly add this to the list of premises; we will call the following proposition *Application Lemma*:

$$\mathrm{AL}_r \triangleq \quad \text{“} \begin{cases} \Gamma \vdash \Pi(x : T).U : \mathsf{rk}_r \\ \Gamma \vdash u : \Pi(x : T).U \\ \Gamma \vdash t : T \\ \vdash \Gamma \\ t, u \in \mathsf{sn} \end{cases} \Rightarrow u\,t \in \mathsf{sn}\text{”}$$

What indexing by r bought us is that now we can try proving this lemma by induction on r. Again, as it always is with strong normalisation of application, the problem is in case where application head is a lambda and a β-reduction takes place. This calls for another lemma, *Substitution Lemma*:

$$\mathrm{SL}_r \triangleq \quad \text{“} \begin{cases} \Gamma, x : T, \Delta \vdash u : U \\ \Gamma \vdash T : \mathsf{rk}_r \\ \Gamma \vdash t : T \\ \vdash \Gamma \\ t, u \in \mathsf{sn} \end{cases} \Rightarrow u[t/x] \in \mathsf{sn}\text{”}$$

A proof of AL_r for all r (which is what we're after) is done by complete mutual induction with SL_r. Thus there are two main parts:

- $(\forall s < r.\ \mathrm{SL}_s) \Rightarrow \mathrm{AL}_r$: by induction on derivations of $u, t \in \mathsf{sn}$. Consider possible reductions of $u\,t$: most of them happen either in u or in t, so that results of reductions from $u\,t$ are strongly normalizing by inductive hypotheses. The only problematic case is when $u = \lambda(x : T').v$ and the reduction is β-reduction. By inversion of typing for u, it should be the case that $T' =_\beta T$; and by inversion of typing for $\Pi(x : T).U$, we get that $T : \mathsf{rk}_s$ and $U : \mathsf{rk}_t$ such that $r = \max(s + 1, t)$. Then, by type stability, $T' : \mathsf{rk}_s$, and furthermore, $s < r$, so we can apply substitution lemma.

- $(\forall s \leq r.\ \mathrm{AL}_s) \Rightarrow \mathrm{SL}_r$. So we need to prove $u[t/x] \in \mathsf{sn}$. We do it by simultaneous induction on derivation of $u \in \mathsf{sn}$ and structural induction on u. As substitution propagates through u, most cases are satisfied by Lemma 4. A funny case is when $u = x$: then, a premise that $t \in \mathsf{sn}$ helps.

  The only problematic case is application $u = u_1\,u_2$; we consider three cases here:

  - $u_1[t/x]$ is atomic — handled by Lemma 4.
  - u_1 is not atomic — by virtue of it being well-typed, it means that $u = (\lambda(x : V).u')v_1 \dots v_k u_2$. All possible reductions from $u[t/x]$ are then handled by induction hypotheses.
  - u_1 is atomic, but $u_1[t/x]$ is not — by virtue of u_1 being well-typed, the only case when this is possible is when $u_1 = x v_1 \dots v_k$, thus $u[t/x] = t(v_1[t/x]) \dots (v_k[t/x])(u_2[t/x])$. From induction hypotheses we know that all $v_i[t/x]$ and $u_2[t/x]$ are s.n.; t is s.n. by premise and its rank is r. Then $tv_1[t/x]$ is s.n. by application lemma for r; but note that rank of $tv_1[t/x]$ is not greater than r so application lemma applies again. Thus, by induction on k, the whole chain of applications is s.n.

□

This is a retelling of a proof that our friend, Aleksandr Levin, got after half-jokingly inputting the following prompt into ChatGPT:

```
Consider the following pure type system:

S = { rk_n | n natural }
Axiom = { (rk_n, rk_0) | n natural }
Rule = { (rk_m, rk_n, rk_max(m+1,n)) | m, n natural }

Is it strongly normalizing or not?
```

After approximately 20 minutes, LLM outputted the proof which we retold above. Looking at it in retrospect, it seems as if we stratified functions by ranks

with intent for such a direct proof to go through; honestly, we didn't think of that and were eagerly trying to adapt well-known approaches for proving normalisation of pure type systems employing either saturated sets or candidates of reducibility, which didn't work because our "hierarchy" of ranks is not well-founded enough because of axioms $\mathsf{rk}_n : \mathsf{rk}_0$. This proof, however, doesn't even bother with it which means that we could pick *any* relation as Ax. Nevertheless, putting all sorts into rk_0 seems the most sensible approach to us since, informally, sort is kind of a constant which cannot be converted into a function.

3.1 Formalizing normalisation of PTS-rk in Rocq

Inductive characterization of the set of strongly normalizing terms follows [1].

4 Computational complexity of PTS-rk

(Proofs in the following sections are not formalized in Rocq: they are relatively simple, non-technical and their formalization would require many times more preliminary work than seems reasonable.)

As requested further, LLM confessed that it was largely inspired by studies on complexity of normalisation for STLC. And indeed, upon taking a closer look at the proof of Theorem 5, one cannot help but notice it actually being a primitive-recursive algorithm for term normalisation in disguise. Or something a tad faster:

Definition 7 (d-bounded terms)**.** A PTS-rk term is d-bounded if it is well-typed in some well-formed context and heads of all redexes in it have rank equal to or below d.

Theorem 6 (Elementary recursive normalisation of bounded PTS-rk)**.** *For each d, there is an elementary recursive algorithm for finding a normal form of d-bounded terms.*

Proof. Consider the following relation of "greedy parallel reduction":

$$x \Rightarrow x \qquad s \Rightarrow s \qquad \frac{T \Rightarrow T' \qquad U \Rightarrow U'}{\Pi(x:T).U \Rightarrow \Pi(x:T').U'} \qquad \frac{T \Rightarrow T' \qquad t \Rightarrow t'}{\lambda(x:T).t \Rightarrow \lambda(x:T').t'}$$

$$\frac{t \Rightarrow (\lambda(x:T).t') \qquad u \Rightarrow u'}{t\,u \Rightarrow t'[u'/x]} \qquad \frac{t \Rightarrow t' \qquad t' \neq \lambda\ldots \qquad u \Rightarrow u'}{t\,u \Rightarrow t'\,u'}$$

It differs from usual parallel reduction since it greedily applies every "many-argument" redex available in a term. For example, $(\lambda(x : X).\lambda(y : Y).\lambda(z : Z).t)\,a\,b\,c$ gets reduced to $t'[a/x][b/y][c/z]$ in a single step. This relation has the following properties:

- $(\Rightarrow) \subseteq (\to_\beta^*)$, so it's several usual reductions baked into one.

- This relation is total and functional.
- This relation can be realised with an elementary recursive algorithm $\mathcal{A}$: traverse the term bottom-up, immediately applying every β-redex which is formed when result of a recursive run of algorithm on application head is a lambda.
- If $t \Rightarrow t'$, then $|t'| \leq (2+\varepsilon)^{\mathcal{O}(|t|)}$.
- If redex heads in t have ranks less than or equal to d and $t \Rightarrow t'$, redex heads in t' have ranks less than d: new redexes appear only after substitution of λ into application head, and rank of this lambda is strictly less than the rank of a redex in which this substitution happened.

Because of the last property, it is enough to run $\mathcal{A}$ for d times in total to normalize a d-bounded term; since input size on k-th run is bounded by k-th tower of exponents, the whole algorithm is elementary-recursive.

To make sure that the corresponding function is total, output 0 if result of an algorithm contains a β-redex. □

Similar result was already proven for STLC since simple types can be classified by ranks as well. And since rank is easily computable for simple types, normalization of arbitrary STLC terms is in TOWER. A little weaker result holds for our dependently typed PTS:

Theorem 7. *Normalisation of* ***closed*** *well-typed PTS-rk terms is in* TOWER.

Proof. By Theorem 6, it is enough to compute some upper bound B on the rank of types appearing in abstractions of a term, then run a normalizer for $(B+1)$-bounded terms. Consider the following elementary recursive functions, written in pseudocode and using a suitable encoding of terms, tuples and lists:

```
maxRankLiteral rk_r       = r
maxRankLiteral Π(x:T).U =
    maxRankLiteral T ⊔ maxRankLiteral U
maxRankLiteral x          = 0
maxRankLiteral λ(x:T).t =
    maxRankLiteral T ⊔ maxRankLiteral t
maxRankLiteral (t u)      =
    maxRankLiteral t ⊔ maxRankLiteral u

traversal _ rk_r       _ = (r , 0 , 0)
traversal Γ Π(x:T).U d =
    let (v^T , r^T , a^T) = traversal Γ T d
        (v^U , r^U , a^U) = traversal (x := v^T, Γ) U d
     in (v^U , suc r^T ⊔ r^U , a^T ⊔ a^U)
traversal Γ x         d = (d , Γ[x] , 0)
traversal Γ λ(x:T).t d =
    let (v^T , r^T , a^T) = traversal Γ T d
```

```
        (v^t , r^t , a^t) = traversal (x := v^T, Γ) t d
     in (v^t , r^t , r^T ⊔ a^T ⊔ a^t)
traversal Γ (t u)    d =
    let (v^t , r^t , a^t) = traversal Γ t d
        (_   , _   , a^u) = traversal Γ u d
     in (v^t , r^t , a^t ⊔ a^u)

estimateRank t =
    let (v^t , r^t , a^t) =
        traversal [] t (maxRankLiteral t)
     in a^t
```

Function `maxRankLiteral` t computes maximal rank which is contained as a subterm in term t. The workhorse of this algorithm, `traversal` $\Gamma\ t\ d$, accepts the list Γ of ranks in context of a term t and a "default rank" d, and outputs the triple (v, r, a), where v is value of t as a rank (if supplied with enough arguments), r is its rank if t is a type (if supplied with enough arguments) and a is an upper bound on rank of types appearing in abstractions which we are trying to compute. The only suspicious case is `traversal` $\Gamma\, x\, d$ where we output d (the "default" rank) as a value of x: couldn't we compute value of x instead, if present? Regrettably, this would require normalization of a term corresponding to x, which is too long; instead we notice that rk_r can be reduced only from terms which already contain it as a subterm, so value of x as a rank can only be rk_r which is already present somewhere in original term. And our result is monotone with respect to the value we output here so it's enough to provide some upper bound on it; thus it is enough to output a maximal rank which occurs in original term as a subterm. □

Note that closedness condition is unavoidable: without it, we wouldn't know ranks of type variables from context and there would be no known bound to the rank of redexes.

Another consequence of Theorem 6 is a natural upper bound for functions *uniformly* expressible in PTS-rk (this result is also similar to the one for STLC).

Definition 8. A family of functions on natural numbers $\mathcal{F}$ is *uniformly* expressible in PTS-rk if there is a *natural number type* $N : \mathsf{rk}_r$, a *zero* $Z : N$, and a *successor* $S : N \to N$ such that for each $(f : \mathbb{N}^k \to \mathbb{N}) \in \mathcal{F}$ there is an *encoding* $F : N \to \ldots \to N \to N$ such that

$$\forall (n_1, \ldots, n_k) \in \mathbb{N}^k : \quad F\,(S^{n_1}\,Z)\,\ldots\,(S^{n_k}\,Z) =_\beta S^{f(n_1,\ldots,n_k)}\,Z$$

Theorem 8. *A family of elementary recursive functions is not uniformly expressible in PTS-rk.*

Proof. By direct diagonalisation argument. Suppose we could encode all elementary recursive functions with $N : \mathsf{rk}_r$, $Z : N$ and $S : N \to N$.

Denote by z and s normal forms of Z and S, respectively. By Theorem 6, normalisation of $(r+1)$-bounded terms $\mathcal{SN}_{r+1}$ is elementary recursive, so the

following function is elementary recursive as well:

$$h(t) = \begin{cases} 1, & \mathcal{SN}_{r+1}(t\,(s^{[t]}\,z)) = z, \\ 0 & \text{otherwise.} \end{cases}$$

(Where $[t] \in \mathbb{N}$ is code of a term t.)

By assumption, it is encodable by $\mathcal{H} : N \to N$; denote by H its normal form. Observe the value of $h(H)$:

$$h(H) = 1 \Leftrightarrow \mathcal{SN}_{r+1}(H\,(s^{[H]}\,z)) = z \Leftrightarrow H\,(s^{[H]}\,z) =_\beta z \Leftrightarrow h(H) = 0$$

First equivalence is by definition of h; second is by virtue of expression under $\mathcal{SN}$ being $(r+1)$-bounded; the last is by definition of h being encoded by $\mathcal{H}$. □

(In the following text, we're going to use some common syntax conventions to make it more lightweight: application associates to the left, so $a\,b\,c$ is understood as $(a\,b)\,c$; in a sequence of abstractions, all λ symbols except the first can be omitted; $T \to U$ is an alias for $\Pi(x : T).U$ (where U doesn't depend on x); $(\to)$ is right-associative, so $T \to U \to V$ is understood as $T \to (U \to V)$. In addition, even though our abstractions are always annotated with a type of an argument, we're going to omit it whenever it's clear from the context.)

A logical next question is then: which functions are computable? Elementary recursive! Though, of course, not in the same sense as before.

Definition 9 (Polymorphic numeral family)**.** For each fixed $j \geq 0$, there's a type of *polymorphic Church numerals* of rank $j + 2$:

$$\mathcal{N}_j \triangleq \Pi(X : \mathsf{rk}_j).(X \to X) \to X \to X;$$

and for each fixed n, there's a corresponding *Church literal* of type $\mathcal{N}_j$:

$$\overline{n} \triangleq \lambda\, X\, f\, x.f^n\, x.$$

($f^n\, x$ means repeated application of f to x: $f(\ldots(f\, x)\ldots)$.)

Theorem 9 (Non-uniform elementary representability)**.** *For each elementary recursive function $f : \mathbb{N}^n \to \mathbb{N}$, there is a* cost c_f *such that for each* base rank j*, there is an* encoding $F_j : \mathcal{N}_{j+c_f} \to \ldots \to \mathcal{N}_{j+c_f} \to \mathcal{N}_j$ *such that*

$$\forall (x_1, \ldots, x_n) \in \mathbb{N}^n : \quad F_j\, \overline{x_1}\, \ldots\, \overline{x_n} =_\beta \overline{f(x_1, \ldots, x_n)}.$$

Proof. As is well-known, simply typed Church numerals admit addition and multiplication. It's still true for our rank-refined version:

$$\begin{array}{lcll} \mathsf{Add}_j & \triangleq & \lambda\, m\, n\, X\, f\, x.m\, X\, f\, (n\, X\, f\, x) & : \mathcal{N}_j \to \mathcal{N}_j \to \mathcal{N}_j \\ \mathsf{Mul}_j & \triangleq & \lambda\, m\, n\, X\, f\, x.m\, X\, (n\, X\, f)\, x & : \mathcal{N}_j \to \mathcal{N}_j \to \mathcal{N}_j \end{array}$$

By substitution-basis theorem, it is enough to additionally provide modulus $x \bmod y$ (where $x \bmod 0 = x$) and binary exponentiation 2^x since substitution, constants and projections are "free" in our calculus.

By Theorem 8, we cannot hope to have both remaining operations represented uniformly. Indeed, naïve import of traditional exponentiation term for Church numerals raises the rank of an exponent:

$$\mathsf{Exp}_j \triangleq \lambda\, m\, n\, (X : \mathsf{rk}_j)\, f\, x.\, n\, (X \to X)\, (\lambda(h : X \to X)(y : X).m\, X\, h\, y)\, f\, x \\ : \mathcal{N}_j \to \mathcal{N}_{j+1} \to \mathcal{N}_j$$

And that's exactly the reason we allowed input and output ranks of Church numerals to differ.

There still remains a problem, however: by allowing ranks to differ, we lost uniformity of substitution. In particular, consider an elementary function $x \mapsto x + 2^x$. To represent such computation using combinators we defined, x has to have both type $\mathcal{N}_j$ (as left addend) and $\mathcal{N}_{j+1}$ (as exponentiation power). Instead of introducing subtyping in the system, we make use of the following family of *numeral-preserving lowerings*:

$$\delta_j \triangleq \lambda\, n\, X\, f\, x.\, n\, (X \to X)\, (\lambda\, h\, y.f\, (h\, y))\, (\lambda\, y.y)\, x : \mathcal{N}_{j+1} \to \mathcal{N}_j$$

As the name implies, the goal of such combinators is to lower the rank of a numeral, but preserve its "value". It is achieved in the following way: given an f and x for lower-ranked type, an original combinator for n is used to build an n-fold composition of f with itself which is then applied to x.

Thus we can lower ranks arbitrarily: $\delta_{b,a} \triangleq \delta_a \circ \ldots \circ \delta_{b-1} : \mathcal{N}_b \to \mathcal{N}_a$. This restores substitution and allows gluing representation of functions together; cost c_f of a function can be computed from any expression that represents it — addition and multiplication cost 0, exponentiation costs 1, operands are merged by taking the maximum of their costs. (Of course, cost is not unique and depends on the way function is represented with an expression.)

All that's left, then, is to represent modulus. General idea is as follows: first, suppose that we're dealing with a simpler problem of taking modulus by a constant modulo, e.g. $\bmod 5$. Notice that there are only 5 possible answers: 0,1,2,3,4. As such, they can be represented by the type which contains exactly 5 elements; Church encoding yields a type of *selectors*:

$$V_5 \triangleq \Pi(X : \mathsf{rk}_j).\, X \to X \to X \to X \to X \to X : \mathsf{rk}_{j+1}$$

Its 5 different canonical values simply pick one of 5 inputs, to each their own.

There is an operation of *cyclic shift* on selectors:

$$c_5 \triangleq \lambda\, s\, X\, x_1\, x_2\, x_3\, x_4\, x_5.\, s\, X\, x_2\, x_3\, x_4\, x_5\, x_1 : V_5 \to V_5$$

Iterating it n times on some pinned "zeroth" selector, we actually get the $(n \bmod 5)$-th selector. All that's left is to read the numeral back:

$$\rho \triangleq \lambda\, s\, X\, f\, x.\, s\, X\, (f^0\, x)\, (f^1\, x)\, (f^2\, x)\, (f^3\, x)\, (f^4\, x) : V_5 \to \mathcal{N}_j$$

Now, in order to model $m \bmod n$ in this vein, we need to *generate* V_n dynamically. Lucky for us, PTS-rk is dependently typed so there is hope.

And indeed, the following construction was produced in discussion with GPT-6 Astra.

We're going to produce a combinator $\mathsf{Mod} : \mathcal{N}_{j+1} \to \mathcal{N}_{j+4} \to \mathcal{N}_j$. Fix parameter $X : \mathsf{rk}_j$ and consider the following signature for abstract $S : \mathsf{rk}_{j+1}$:

$$\begin{array}{ll} K : X \to S, & z : S, \\ L : (X \to S) \to X \to S, & c : S \to S, \\ \rho : S \to (X \to X) \to X \to X, & r : (X \to X) \to (X \to X). \end{array}$$

Intention is that S is simply-typed V_q for X, z is its "zero", K is a "zero" for V_{q+1}, c is a cyclic shift for V_q, L is a cyclic shift for V_{q+1}, ρ converts a selector into a numeral, and r is the result of modulus computed using structure from the previous step (this is similar to the trick used to define a predecessor function).

A callback accepting such structure and returning X has the following type:

$$\begin{aligned} H_X \triangleq \Pi(S : \mathsf{rk}_{j+1}).\ (X \to S) \to S \to ((X \to S) \to X \to S) \to (S \to S) \to \\ (S \to (X \to X) \to X \to X) \to ((X \to X) \to X \to X) \to X \quad : \mathsf{rk}_{j+3} \end{aligned}$$

A type of "records" is thus $P_X \triangleq H_X \to X : \mathsf{rk}_{j+4}$, with a constructor

$$\langle S, K, z, L, c, \rho, r \rangle \triangleq \lambda h.\, h\, S\, K\, z\, L\, c\, \rho\, r.$$

As an initial P_X, we build a structure for V_1 and an r which behaves like $m \bmod 0 = m$:

$$\begin{array}{ll} S_0 \triangleq X \to X, & \\ K_0 \triangleq \lambda(a : X)(b : X).a, & z_0 \triangleq \lambda(a : X).a, \\ L_0 \triangleq \lambda(t : X \to S_0)(a : X)(b : X).t\, b\, a, & c_0 \triangleq \lambda(s : S_0).s, \\ \rho_0 \triangleq \lambda(s : S_0)(g : X \to X)(a : X).s\, a, & r_0 \triangleq \delta_j\, m\, X. \end{array}$$

Pack them into $p_0 \triangleq \langle \ldots \rangle$.

Then, given a structure for q, a structure for $q + 1$ is:

$$\begin{array}{ll} S^+ \triangleq X \to S, & \\ K^+ \triangleq \lambda(a : X)(b : X).Ka, & z^+ \triangleq K, \\ L^+ \triangleq \lambda(t : X \to S^+)(a : X)(b : X).L\,(t\, b)\, a, & c^+ \triangleq L, \\ \rho^+ \triangleq \lambda(t : S^+)(g : X \to X)(a : X).\rho\,(ta)\, g\,(ga), & r^+ := \rho\,(m\, S\, c\, z). \end{array}$$

A transition term can be written like this:

$$T \triangleq \lambda(p : P_X)(h : H_X).p\,(\lambda\, S\, K\, z\, L\, c\, \rho\, r.\, h\, S^+\, K^+\, z^+\, L^+\, c^+\, \rho^+\, r^+) : P_X \to P_X$$

Therefore, the following term computes modulus with expected behaviour:

$$\begin{aligned} \mathsf{Mod}_j \triangleq \lambda(m : \mathcal{N}_{j+1})(n : \mathcal{N}_{j+4})(X : \mathsf{rk}_j)(f : X \to X)(x : X). \\ n\, P_X\, T\, p_0\, (\lambda _____ r.r\, X\, f\, x). \end{aligned}$$

□

Two questions remain open: a) whether there are non-elementary functions which are non-uniformly computable in PTS-rk and b) whether there are any non-polynomial functions which are uniformly computable in PTS-rk.

5 Set-theoretical model of PTS-rk

Now, studying computational properties of a calculus is great, but a logic is only useful when it can tell right from wrong. In this section, we're going to provide a nontrivial set-theoretical semantics for PTS-rk with respect to which it is sound. In particular, $[\![\Pi(X : \mathsf{rk}_j).X]\!]$ is going to be an empty set, as expected.

We're going to need an unusual assumption however:

Theorem 10. *PTS-rk has a sound model in ZF+GCH.*

Proof. Produced in discussion with GPT 6 Astra.

Take distinct codes c_κ for every cardinal below $\aleph_\omega$ and distinct additional codes s_j for $j \in \mathbb{N}$. Define

$$U_i = \{c_\kappa : \kappa \leq \aleph_i\} \cup \{s_j : j \in \mathbb{N}\}, \qquad \mathrm{El}(c_\kappa) = \kappa, \quad \mathrm{El}(s_j) = U_j.$$

That is,

$$\begin{aligned}
&U_0 = \{c_0, c_1, \ldots, c_\omega = c_{\aleph_0}, s_0, s_1, \ldots\},\\
&U_1 = \{c_0, c_1, \ldots, c_{\aleph_0}, c_{\aleph_1}, s_0, s_1, \ldots\},\\
&\ldots\\
&U_k = \{c_0, c_1, \ldots, c_{\aleph_0}, c_{\aleph_1}, \ldots, c_{\aleph_k}, s_0, s_1, \ldots\}\\
&\ldots
\end{aligned}$$

(Note that $x \in U_i \Rightarrow |\mathrm{El}(x)| \leq \aleph_i$ since all U_i are countable.)

Interpretation then goes as follows:

- Interpret rk_i by s_i. The sort axiom holds since $s_i \in U_0 = \mathrm{El}(s_0)$.
- If $a \in U_m$ and $b(x) \in U_n$ for every $x \in \mathrm{El}(a)$, the ordinary dependent function set has cardinality at most

$$\left| \prod_{x \in \mathrm{El}(a)} \mathrm{El}(b(x)) \right| \leq \aleph_n^{\aleph_m} \leq \aleph_{\max(m+1,n)}.$$

  (The last inequality follows from GCH since alephs have finite indices.)

  Thus, for all such a, b we can choose a cardinal $\kappa_{\Pi(x:a).b(x)}$ (whose code lies in $U_{\max(m+1,n)}$) along with a bijection

$$f : \left(\prod_{x \in \mathrm{El}(a)} \mathrm{El}(b(x)) \right) \simeq \kappa_{\Pi(x:a).b(x)}$$

The eligible pairs (κ, f) form a set, so ordinary choice (which holds in ZF+GCH) suffices. Make each choice once for each (a, b), independently of a typing derivation or rank bound.

- Abstraction and application are modeled by f and f^{-1} respectively, as defined above.
- Contexts are interpreted by valuations.

It is easy to check that typing rules are respected by such interpretation, making it a sound model. □

A few corollaries:

- Indeed, the product interpreting $\Pi Z : \mathsf{rk}_0.Z$ ranges over U_0, which contains c_0, whose decoding is $\varnothing$. That product is therefore empty.
- A system with rank cumulativity $t : \mathrm{rk}_j \Rightarrow t : \mathrm{rk}_{j+1}$ is sound with respect to this model; normalization of bounded terms remains elementary as in Theorem 6.

Note that usage of GCH is unavoidable in this model to validate the rules. We could try building a model in ZFC using $\beth$-s instead of $\aleph$-s, but that would break an assumption that U_i are countable, which would falsify axioms $\mathsf{rk}_n : \mathsf{rk}_0$. That's not a huge problem however since ZF+GCH is interpretable in ZFC (in contrast to ZFC + inaccessible cardinals which is not).

6 Proof-theoretic strength of PTS-rk

7 Conclusion

References

[1] Felix Joachimski and Ralph Matthes. "Short proofs of normalization for the simply-typed λ-calculus, permutative conversions and Gödel's T". In: *Archive for Mathematical Logic* 42.1 (2003), pp. 59–87. DOI: 10.1007/s00153-002-0156-9. URL: https://doi.org/10.1007/s00153-002-0156-9.